Карта проекта

Документ для защиты. Читается за пятнадцать минут и даёт целостную картину: что делает инструмент, из чего он собран, как ходят данные, какие решения приняты и почему.

1. Что это

«Лаборатория протоколов» — локальное десктопное приложение для исследования недокументированного бинарного обмена поверх TCP. Инженер открывает дамп трафика, выбирает сессию, восстанавливает поток байтов, описывает структуру сообщений декларативными правилами, применяет их ко всему корпусу, получает контрпримеры и уточняет правило.

- **Вход.** Файл `PCAP` или `PCAPNG`, снятый на стенде.
- **Выход.** Нормализованный захват (JSON), результат применения правила (JSON), отчёт с контрпримерами, переносимый проект-контейнер, отчёт в Markdown и HTML.
- **Три инварианта.** 1. **Наблюдение ≠ гипотеза.** Факт и предположение всегда различимы: поле помечается `hypothesis: true`, статус `matched` означает «согласуется с наблюдением», а не «доказано». 2. **Контрпример обязателен.** Совпадение на примерах не является доказательством. Правило проверяется на корпусе, и найденный контрпример привязывается к конкретным байтам. 3. **Воспроизводимость.** Изменение правила не меняет исходные байты; каждый результат несёт версию правила и версию контракта, а проверка повторяема из командной строки.

2. Структура репозитория

```
src/                      исходный код, семь модулей (см. раздел 3)
  capture/                чтение pcap, сессии, реассемблирование, provenance
  protocol/               фрейминг, правила, движок, экспорт, CLI
  hypothesis/             корпус, контрпримеры, версии, метрики, alternatives
  project/                переносимый проект: манифест, пайплайн, CLI
  report/                 отчёты в Markdown и HTML
  ui/                     десктопный интерфейс на PySide6
  web/                    необязательный локальный веб-интерфейс FastAPI + HTMX
tests/                    тесты по модулям и сквозные
  capture/ protocol/ hypothesis/ project/ report/ ui/ web/  тесты по стадиям
  integration/            стыковка стадий
  corpus/                 корпуса: синтетика, дефекты, реальные дампы (real/)
  fixtures/               мелкие дампы, собранные генератором
docs/                     документация: контракты, API, метрики, защита
  schemas/                JSON Schema контрактов (capture, rule, result)
  screenshots/            снимки интерфейса для документации
examples/                 примеры правил (thermostand_request/response, corpus_rule)
scripts/                  генераторы фикстур и корпусов, сборки, бенчмарк
packaging/                упаковка: иконки, службы, метаданные сборок
presentation/             слайды (Markdown, PDF, PPTX) и вопросы защиты
assets/                   шрифты и иконки приложения
build/ dist/              артефакты сборки (не входят в исходники)
```

3. Модули и их роли

`src/capture/` — R1, движок захвата

Читает `PCAP` и `PCAPNG` на чистом Python и группирует TCP-пакеты в сессии по пятикортежу с учётом повторного использования портов. Восстанавливает оба направления по номерам последовательности и записывает `provenance` для каждого диапазона байт. Пропуски не заполняются нулями: они попадают в `diagnostics` как `gap`. Пересечения и ретрансмиссии размечаются отдельными диагностиками.

`src/protocol/` — R3, фрейминг, правила, движок

Режет поток на сообщения четырьмя стратегиями: `length_prefixed`, `fixed_size`, `marker_based`, `manual`. Описывает раскладку сообщения декларативным правилом (смещения, типы, порядок байт) с частичным описанием. Применяет правило к потоку и выдаёт по каждому полю значение, диапазон байт и статус. Экспортирует правило во внешние формы (Kaitai Struct, автономный Python-парсер).

`src/hypothesis/` — R4, корпус и контрпримеры

Проверяет правило на всём корпусе и сводит результат в отчёт: сколько сообщений `matched`, `mismatched`, `incomplete`, `ambiguous`, `uncovered`, `not_applicable`. Находит противоречия и привязывает каждое к байтам. Версионирует правила: старая проверка помечается `outdated`, повторная даёт сравнение версий. Строит альтернативные разборы и считает метрики качества. Связывает сообщение с журналом исследователя (`journal.py`) и с соседним сообщением противоположного направления (`pairing.py`): ход получает оценку уверенности и перечень наблюдений, на которых она основана.

`src/project/` — R5, переносимый проект

Собирает исследование в один каталог-контейнер: манифест `manifest.json` с относительными путями и контрольными суммами `sha256`, вложенные захваты, правила, результаты и журнал в `SQLite`. Каталог можно скопировать на другую машину без правки путей. Пайплайн прогоняет стадии от дампа до отчёта одной командой; проект экспортируется и импортируется архивом.

`src/report/` — R6, отчёты

Сводит результаты проверки в человекочитаемый отчёт на русском: сводка по статусам, таблица сообщений, контрпримеры с диапазонами байт. Выдаёт Markdown для чтения и HTML для печати из одного и того же представления.

`src/ui/` — десктопный интерфейс

Окно на PySide6: дерево сессий, шестнадцатеричный просмотр с подсветкой provenance, панель правил, сравнение версий, список гипотез. Тёмная и светлая темы, проверка контраста, шрифты для кириллицы.

`src/web/` — необязательный веб-интерфейс

Локальный сервер на FastAPI с разметкой HTMX для тех же стадий. Запускается по желанию, не требуется для основной работы и не заменяет десктопное приложение.

4. Поток данных

```
PCAP / PCAPNG
     │  src.capture.cli
     ▼
нормализованный захват (JSON)          сессии, направления, provenance, diagnostics
     │  src.protocol.cli apply
     ▼
результат применения правила (JSON)    сообщения, поля, статусы
     │  src.protocol.cli verify
     ▼
отчёт проверки на корпусе              контрпримеры, метрики, сравнение версий
     │  src.project.cli pipeline
     ▼
переносимый проект (manifest + sha256)
     │  src.report
     ▼
отчёт в Markdown и HTML
     │
     ▼
десктопный интерфейс (src.ui) или локальный веб (src.web)
```

Каждая стрелка — это файл-контракт: стадия читает JSON предыдущей стадии и пишет свой. Стадии можно запускать по отдельности и перепроверять независимо.

5. Контракты JSON

Нормализованный захват — выход `src/capture`, вход `src/protocol`:

```
{
  "contract_version": 1,
  "capture_id": "sha256:<64 hex>",
  "source_file": "captures/cap1.pcapng",
  "sessions": [{
    "session_id": "s1",
    "endpoints": [{"ip": "...", "port": 0}, {"ip": "...", "port": 0}],
    "directions": {
      "A_to_B": {
        "bytes_b64": "...",
        "provenance": [{"offset": 0, "length": 4, "packet_index": 42, "seq": 1000, "ts":
"..."}],
        "diagnostics": [{"type": "gap", "offset": 100, "length": 8}]
      }
    }
  }]
}
```

`bytes_b64` содержит только наблюдаемые байты. Пропуски не заполняются нулями, а описываются диагностикой `gap`.

Результат применения правила — выход `src/protocol`, вход `src/hypothesis` и `src/report`:

```
{
  "contract_version": 1,
  "rule_id": "r1",
  "rule_version": 3,
  "capture_id": "...",
  "messages": [
    {"offset": 0, "length": 16, "fields": {"cmd": 4, "value": 21}, "status": "matched"}
  ],
  "summary": {"matched": 205, "mismatched": 12, "unknown": 23, "uncovered": 0}
}
```

Полные определения и схемы: CONTRACT.md, schemas/capture.schema.json, schemas/rule.schema.json, schemas/result.schema.json.

6. CLI

Команда	Назначение
`python -m src.capture.cli --pcap cap.pcapng --out normalized.json`	Нормализовать дамп в контракт сессий. Ключи: `--source-name`, `--verify-checksums`, `--stream`, `--chunk-size`, `--wireshark-pcap`.
`python -m src.protocol.cli apply`	Применить одно правило к одному направлению.
`python -m src.protocol.cli verify`	Проверить одно правило на всех потоках захвата.
`python -m src.protocol.cli diff`	Сравнить два правила и два отчёта проверки.
`python -m src.protocol.cli metrics`	Посчитать метрики качества по отчёту.
`python -m src.protocol.cli correlate`	Сопоставить сообщения потока с журналом действий.
`python -m src.protocol.cli export`	Выгрузить правило в `.ksy` Kaitai Struct или Python-парсер.
`python -m src.protocol.cli alternatives`	Построить альтернативные разборы.
`python -m src.project.cli pipeline`	Прогнать стадии от дампа до отчёта одной командой.
`python -m src.project.cli create` / `open`	Создать проект или показать его манифест.
`python -m src.project.cli add-capture`	Скопировать захват в проект.
`python -m src.project.cli export` / `import`	Выгрузить проект в zip или загрузить его обратно.
`python -m src.ui.main_window`	Запустить десктопное приложение.
`python -m src.web.app`	Запустить необязательный локальный веб-интерфейс.

Пример полного цикла:

```
python -m src.capture.cli --pcap tests/corpus/real/research/01_configuration.pcap --out
normalized.json
python -m src.protocol.cli apply --rule examples/thermostand_request_v2.json --capture
normalized.json --out result.json
python -m src.protocol.cli verify --rule examples/thermostand_request_v2.json --capture
normalized.json --out report.json
```

7. Ключевые технические решения

1. **dpkt, а не PyShark.** `dpkt` читает и классический `pcap`, и `pcapng` на чистом Python, без внешних процессов. PyShark отклонён: он вызывает `tshark`, а инструмент должен работать офлайн и ставиться одним `pip`.
2. **`length_covers: payload` по умолчанию.** Для length-prefixed фрейминга счётчик длины по умолчанию покрывает только полезную нагрузку. Такой смысл встречается чаще, а обратный вариант выражается явно; поведение зафиксировано и описано в коде и документации.
3. **Алгоритмические альтернативы вместо ML-моделей.** Разбор идёт детерминированно и офлайн: каждый вывод объясним — видно, из каких байт он получен, — а один и тот же поток байт даёт один и тот же результат. Требование объяснимости и воспроизводимости.
4. **TCP-реассемблирование.** Байты упорядочиваются по номеру последовательности с учётом переполнения 32-битного счётчика. Идентичная ретрансмиссия не добавляет байт, а расширяет диапазон provenance. Конфликтующее пересечение сохраняет первые байты и помечается диагностикой `ambiguity` с обеими версиями. Пропуск даёт диагностику `gap`.
5. **Provenance.** Каждый диапазон байт несёт `packet_index`, `seq` и `ts`. Это позволяет показать, из какого пакета пришли байты, и отличить реальные данные от восстановленных или конфликтующих.
6. **Переносимый проект.** Каталог-контейнер с манифестом `manifest.json`, относительными путями и контрольными суммами `sha256` каждого артефакта. Каталог переносится на другую машину как есть; целостность проверяется по суммам.
7. **Экспорт правила.** Правило выгружается как описание Kaitai Struct (`.ksy`) и как автономный Python-парсер без зависимости от проекта. То, что экспорт проверить не может (контрольные суммы, совпадение с гипотезой), помечается комментарием, а не выдаётся за проверку.

8. Тестирование

- **957 тестов** в дереве репозитория, все проходят; в чистой распаковке архива **793 проходят, 1 пропускается** (пропуск — проверка бита исполняемости: zip не хранит режим файла; счёт 793 относится к замороженному архиву `v1.0.0`).
- Покрыты: фрейминг всеми четырьмя стратегиями; применение правила и все статусы; контрпримеры и противоречия; версионирование и пометка `outdated`; пропуски и неоднозначности; сообщение, разбитое между пакетами; сборка архива; стадии проекта; отчёты; интерфейс и веб.
- **CI:** пять проверок на каждый push — `test`, `ui`, `code-quality`, `dependency-audit` и сборка. Все зелёные.
- Локальные проверки: `pytest tests/ -q`, `ruff check . --select "E4,E7,E9,F" --ignore E501`.

9. Ограничения

Вне области технического задания:

- **IPv6** — разбирается наравне с IPv4: TCP-сессии поверх IPv6 дают потоки, контрольные суммы проверяются при `--verify-checksums`. Фрагментация IPv6 собирается; при перекрытии датаграмма отбрасывается по RFC 5722.
- **IP-фрагментация** — собирается для обеих версий до передачи в TCP-слой. Пропуск фиксируется как `ip_fragment_incomplete`, перекрытие — как `ip_fragment_overlap`, сборка — как `ip_fragment_reassembled`.
- **Шифрование** — зашифрованный трафик разбирается как байты, поля не интерпретируются.
- **Live capture** — работа идёт с готовыми файлами, сеть не слушается.

Эти границы заданы намеренно: инструмент исследует дамп, а не ведёт съём.

10. Ссылки

- README.md — обзор и быстрый старт.
- ARCHITECTURE.md — подход, модули, форматы, зависимости.
- REPORT.md — отчёт о работе.
- docs/CONTRACT.md — JSON-контракты между стадиями.
- docs/REAL_CORPUS_ANALYSIS.md — разбор реального корпуса.
- docs/METRICS.md — числа с указанием источника.
- docs/ARCHITECTURE_DECISIONS.md — решения и их основания.
- docs/PROTOCOL_API.md — API движка правил и гипотез.
- docs/CAPTURE_API.md — API движка захвата.
- docs/DEMO_CHEATSHEET.md — одностраничник для защиты.
- docs/BEYOND_SCOPE.md — что реализовано сверх ТЗ и что честно не является заслугой.